Lab 9A – Distributed Machine Learning: Foundations

CS4074 Big Data Analytics | Effat University, Computer Science Department

Spring 2025 | Estimated time: 2.5 hours | Total: 68 points

→ **Continue with Lab 9B (Advanced Systems) after completing this lab**

Student Name	Student ID	Section
Farah Aburayya - Dalia Guail - Asma Alsakkaf	S21107290- S21207452 - S21107405	1

By completing this lab, you will be able to:

- Quantify the single-node memory and compute wall through profiling experiments
- Apply Amdahl's Law to reason about the limits of distributed speedup
- Build a full Spark MLlib Pipeline with feature engineering and model evaluation
- Measure the effect of partition count on distributed training throughput
- Implement and analyse core collective communication primitives from scratch
- Compare broadcast join vs. shuffle join in Spark and explain the performance gap

Part	Topic	Points
Part 1	The Single-Node Wall: Memory Profiling & Amdahl's Law	20
Part 2	Data Parallelism with Spark MLlib Pipelines	30
Part 3	Communication Primitives: Broadcast, Reduce, All-Reduce	18
Total		**68**

You must have **PySpark 3.x** and **Python 3.8+** installed. Parts 1 and 3 use only standard Python libraries (numpy, time, sys). Part 2 requires a running Spark session and HDFS access. No GPU is required.

Part 1 | The Single-Node Wall: Memory Profiling & Amdahl's Law

A single GPU today carries at most 80 GB of HBM (e.g., NVIDIA A100). A single transformer layer of GPT-3 (175 B parameters) requires roughly 350 GB in full float32 precision — more than four A100s combined. This part makes that wall *concrete*: you will profile memory growth and training time as dataset size scales, then use Amdahl's Law to reason about the theoretical ceiling of distributed speedup.

Task 1.1: Dataset Generation and Memory Profiling [10 pts]

Write a Python script that generates synthetic classification datasets of increasing size and records the memory footprint of the feature matrix at each scale.

Starter Code — memory_wall.py

```python
import numpy as np
import sys
import time

sizes = [10_000, 100_000, 500_000, 1_000_000, 5_000_000]
n_features = 50

print(f"{'Rows':>12} {'Features':>10} {'Matrix MB':>12} {'Gen Time (s)':>14}")
print("-" * 55)

for n in sizes:
    t0 = time.time()
    X = np.random.randn(n, n_features).astype(np.float32)
    y = (X[:, 0] + X[:, 1] > 0).astype(np.int32)
    elapsed = time.time() - t0

    mb = X.nbytes / (1024 ** 2)
    print(f"{n:>12,} {n_features:>10} {mb:>12.2f} {elapsed:>14.4f}")
    del X, y    # free before next iteration
```

Your Tasks

1. Run the script and paste the printed table in your report.
2. Add a column recording sys.getsizeof(X) in bytes and compare it with X.nbytes. Explain any discrepancy.
3. At which row count does generation time first exceed 1 second on your machine?
4. **Predict** (with arithmetic): switching to float64 instead of float32, how large would the 5 M-row matrix be in MB? Verify by changing the dtype.

[10 pts]

1) Printed Table

Rows	Features	Matrix MB	sys bytes	Gen Time (s)
10,000	50	1.91	2000128	0.0173
100,000	50	19.07	20000128	0.0957
500,000	50	95.37	100000128	1.2708
1,000,000	50	190.73	200000128	1.8161
5,000,000	50	953.67	1000000128	13.8043

2) sys.getsizeof(X) vs X.nbytes

X.nbytes represents the exact memory used by the data stored in the NumPy array.
sys.getsizeof(X) includes additional overhead from the Python object itself (metadata and structure).

Therefore, sys.getsizeof(X) is slightly larger than X.nbytes due to Python object overhead.

3) When does time exceed 1 second

The generation time first exceeded 1 second at **500,000 rows**, where the execution time was approximately **1.27 seconds**.

4) float64 size prediction

Switching from float32 to float64 doubles the memory usage because each value increases from 4 bytes to 8 bytes.

For 5,000,000 rows and 50 features:
Total values = 250,000,000

- float32 ≈ 953.67 MB
- float64 ≈ 1907.35 MB

This matches the experimental result, confirming that memory usage doubles when using float64.

Task 1.2: Amdahl's Law Simulation

Amdahl's Law bounds the theoretical speedup achievable by parallelisation:

$$S(p) = \frac{1}{f_s + \frac{1 - f_s}{p}}$$

where f_s is the serial fraction (driver overhead, shuffle coordination) and p is the number of parallel workers.

Starter Code — amdahl.py

```python
import numpy as np

serial_fractions = [0.05, 0.10, 0.20, 0.50]
workers = [1, 2, 4, 8, 16, 32, 64]

print(f"{'Workers':>8}", end="")
for fs in serial_fractions:
    print(f"  fs={fs:.0%}", end="")
print()
print("-" * (8 + 8 * len(serial_fractions)))

for p in workers:
    print(f"{p:>8}", end="")
    for fs in serial_fractions:
        speedup = 1 / (fs + (1 - fs) / p)
        print(f"  {speedup:>6.2f}", end="")
    print()
```

Your Tasks

1. Run the script and record the speedup table.
2. A Spark job has $f_s = 0.15$ (driver overhead + shuffle bookkeeping). What is the maximum theoretical speedup regardless of how many workers you add? Show the algebraic limit $\lim_{p\to\infty} S(p)$.
3. Using $f_s = 0.10$, how many workers are needed to achieve 80 % of the theoretical maximum? Show your calculation.
4. **Why does doubling workers beyond a certain point yield diminishing returns?** Write 3–4 sentences connecting your answer to Spark's shuffle and communication overhead.

1) Speedup Table

Workers	fs=5%	fs=10%	fs=20%	fs=50%
1	1.00	1.00	1.00	1.00
2	1.90	1.82	1.67	1.33
4	3.48	3.08	2.50	1.60
8	5.93	4.71	3.33	1.78
16	9.14	6.40	4.00	1.88
32	12.55	7.80	4.44	1.94
64	15.42	8.77	4.71	1.97

2) Maximum theoretical speedup (fs = 0.15)

Using Amdahl's Law:

$S(p) = 1 / (fs + (1 - fs)/p)$

Taking the limit as $p \to \infty$:

$\lim_{p\to\infty} S(p) = 1 / fs$

$= 1 / 0.15$
$= 6.67$

So, the maximum theoretical speedup is **6.67**.

3) Workers needed for 80% of max (fs = 0.10)

Maximum speedup:

Smax = 1 / 0.10 = 10

80% of maximum:

0.8 × 10 = 8

Solve:

1 / (0.10 + 0.90/p) = 8

0.10 + 0.90/p = 0.125

0.90/p = 0.025

p = 36

So, approximately **36 workers** are needed.

4) Why diminishing returns

Doubling the number of workers does not continue to double performance because the serial portion of the computation cannot be parallelized. In Spark, increasing the number of workers also increases shuffle and communication overhead between nodes. Data must be transferred across the network, which adds latency and coordination cost. As a result, beyond a certain point, the overhead outweighs the benefits of parallelism, leading to diminishing returns.

Part 2 | Data Parallelism with Spark MLlib Pipelines

Spark MLlib implements **data parallelism** natively: the dataset is partitioned across executors, each executor computes local gradients (or sufficient statistics), and the driver aggregates results via a reduce operation. In this part you build a complete MLlib Pipeline, scale the dataset, and measure how distributed training behaves under increasing load.

Task 2.1: SparkSession and Dataset Preparation [8 pts]

Starter Code

```python
from pyspark.sql import SparkSession
import pyspark.sql.functions as F
import time

spark = SparkSession.builder \
    .appName("Lab9A-Foundations") \
    .config("spark.driver.memory", "4g") \
    .config("spark.sql.shuffle.partitions", "8") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")
print("Spark version:", spark.version)
```

We will use the **UCI Adult Income** dataset (Census Income, 48 842 records). Download it and upload to HDFS:

Terminal — HDFS Upload

```bash
wget https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data \
  -O adult.csv
hdfs dfs -mkdir -p /user/student/lab9
hdfs dfs -put adult.csv /user/student/lab9/adult.csv
hdfs dfs -ls /user/student/lab9/
```

Your Tasks

1. Load the CSV into a Spark DataFrame with meaningful column names: `age`, `workclass`, `fnlwgt`, `education`, `education_num`, `marital_status`, `occupation`, `relationship`, `race`, `sex`, `capital_gain`, `capital_loss`, `hours_per_week`, `native_country`, `income`. Print the schema.
2. Show the first 5 rows and the total record count.
3. Create a binary label column `label`: 1 if `income` is `">50K"`, else 0.
4. Report the class distribution (count and percentage per class). Is the dataset balanced?

1) Load CSV and print schema

The CSV file was successfully loaded into a Spark DataFrame with meaningful column names. The schema includes both numerical and categorical columns such as age, workclass, education, and income. The schema was printed using `df.printSchema()`.

2) First 5 rows and total count

The first 5 rows of the dataset were displayed using `df.show(5)`.

The total number of records in the dataset is **32,561**.

3) Create binary label

A binary label column was created where:

- label = 1 if income is ">50K"
- label = 0 otherwise

This was implemented using a conditional transformation on the income column.

4) Class distribution and balance

Class distribution:

- Class 0: 24,720 records (75.92%)
- Class 1: 7,841 records (24.08%)

The dataset is **not balanced**, since the majority of the data belongs to class 0.

Task 2.2: Building a Full MLlib Pipeline [14 pts]

Starter Code

```python
from pyspark.ml import Pipeline
from pyspark.ml.feature import (StringIndexer, OneHotEncoder,
                                VectorAssembler, StandardScaler)
from pyspark.ml.classification import (LogisticRegression,
                                       RandomForestClassifier)
from pyspark.ml.evaluation import BinaryClassificationEvaluator

categorical_cols = ["workclass", "education", "marital_status",
                    "occupation", "relationship", "race",
                    "sex", "native_country"]
numeric_cols     = ["age", "fnlwgt", "education_num", "capital_gain",
                    "capital_loss", "hours_per_week"]

# Stage 1: StringIndexer per categorical column
indexers = [StringIndexer(inputCol=c, outputCol=c+"_idx",
                          handleInvalid="keep")
            for c in categorical_cols]

# Stage 2: OneHotEncoder
encoders = [OneHotEncoder(inputCol=c+"_idx", outputCol=c+"_ohe")
            for c in categorical_cols]

# Stage 3: VectorAssembler
assembler = VectorAssembler(
    inputCols=[c+"_ohe" for c in categorical_cols] + numeric_cols,
    outputCol="raw_features")

# Stage 4: StandardScaler
scaler = StandardScaler(inputCol="raw_features", outputCol="features",
                        withMean=False, withStd=True)

# Stage 5: Classifier
lr = LogisticRegression(featuresCol="features", labelCol="label",
                        maxIter=20, regParam=0.01)

pipeline = Pipeline(stages=indexers + encoders + [assembler, scaler, lr])
```

Your Tasks

1. Split the data 80/20 (train/test) with seed=42. Train the pipeline and record wall-clock time.
2. Evaluate on the test set using BinaryClassificationEvaluator with areaUnderROC and areaUnderPR. Report both metrics.
3. Replace LogisticRegression with RandomForestClassifier (100 trees). Retrain and compare AUC-ROC. Which model performs better?
4. Use pipeline.stages to print the ordered stage list. How many stages does your pipeline have? Why does stage order matter?
5. **Examine the execution plan**: call train_df.explain(True) and identify at least two physical operators that indicate distributed execution.

1) Train/test split and training time

The dataset was split into 80% training and 20% testing using a seed of 42.
The pipeline was trained successfully, and the recorded training time was **18.269 seconds**.

2) Evaluation metrics

The model was evaluated on the test set using BinaryClassificationEvaluator:

- AUC-ROC = **0.9034**
- AUC-PR = **0.7580**

3) Random Forest comparison

After replacing LogisticRegression with RandomForestClassifier (100 trees):

- Random Forest AUC-ROC = **0.8893**

Logistic Regression achieved a higher AUC-ROC score (**0.9034**) compared to Random Forest (**0.8893**).
Therefore, **Logistic Regression performs better** on this dataset.

4) Pipeline stages

The pipeline consists of **19 stages**.

This includes:

- Multiple StringIndexer stages
- Multiple OneHotEncoder stages
- VectorAssembler
- StandardScaler
- Classifier

Stage order matters because each transformation depends on the output of the previous stage. For example, categorical variables must be indexed before encoding, and all features must be assembled before being passed to the model.

5) Execution plan

From the execution plan (`train_df.explain(True)`), the following physical operators were observed:

- FileScan
- Project
- Sort

These operators indicate distributed execution, as Spark reads data from distributed storage and applies transformations across partitions.

Task 2.3: Partition Scaling Experiment [8 pts]

Starter Code

```python
partition_counts = [1, 2, 4, 8, 16]
results = []

for n_parts in partition_counts:
    df_repartitioned = train_df.repartition(n_parts)
    t0 = time.time()
    model = pipeline.fit(df_repartitioned)
    elapsed = time.time() - t0
    results.append((n_parts, round(elapsed, 3)))
    print(f"Partitions: {n_parts:>3} | Training time: {elapsed:.3f}s")
```

Your Tasks

1. Record training time for each partition count in a table.
2. At what partition count is training time minimised on your machine?
3. Why might 16 partitions be *slower* than 8 on a 4-core machine? Explain in terms of task scheduling overhead.
4. What does df.rdd.getNumPartitions() return before and after repartitioning?

---

1) Training time table

Partitions	Training Time (s)
1	6.958
2	6.878
4	6.677
8	9.169
16	13.321

2) Minimum training time

The training time is minimized at **4 partitions**, where the execution time is **6.677 seconds**.

3) Why 16 partitions is slower

Using 16 partitions on a 4-core machine creates more tasks than the available CPU cores. This leads to increased task scheduling overhead and context switching. Spark must manage and coordinate more tasks, which adds extra execution cost. As a result, performance decreases instead of improving.

4) Number of partitions before and after

Before repartitioning, `df.rdd.getNumPartitions()` returns **1**.
After applying `repartition(n)`, it returns **n**, matching the number of partitions specified.

Part 3 | Communication Primitives: Broadcast, Reduce, All-Reduce

Mental Model

Broadcast = **One speaks → Everyone listens**

Reduce = **Everyone contributes → One collects**

AllReduce = **Everyone contributes + Everyone receives — no central server**

Think of **AllReduce** like a group vote where every person both casts *and* receives the final tally simultaneously. The ring variant does this by passing partial results around a circle, so no single node is a bottleneck.

Before workers can aggregate gradients, they must execute **collective communication operations**. In this part you implement four key primitives using plain Python and NumPy — the same operations that underlie NCCL, Gloo, and Spark's shuffle service. Understanding their cost is essential for designing efficient distributed training systems.

Task 3.1: Implementing Collective Primitives [12 pts]

Starter Code — collectives.py

```python
import numpy as np
import time

def simulate_broadcast(source_data, n_workers):
    """
    Worker 0 holds source_data and sends it to all other workers.
    Returns: list of length n_workers, each entry is a copy of source_data.
    """
    # TODO: implement
    pass

def simulate_reduce(worker_data_list, reduce_fn=np.add):
    """
    Each worker holds a local array. Reduce (sum) at worker 0.
    Returns: the aggregated result at worker 0.
    """
    # TODO: implement
    pass

def simulate_allreduce_naive(worker_data_list):
    """
    Naive all-reduce: worker 0 acts as parameter server --
    collect from all, aggregate, broadcast result back.
    Returns: list of length n_workers, each holding the global sum.
    """
    # TODO: use simulate_reduce + simulate_broadcast
    pass
```

```python

def simulate_allreduce_ring(worker_data_list):
    """
    Ring all-reduce (two-phase):
      Phase 1 Scatter-Reduce: n-1 rounds, each worker sends a chunk
        to its right neighbour and accumulates the incoming chunk.
      Phase 2 All-Gather: n-1 rounds, propagate reduced chunks.
    Returns: list of length n_workers, each holding the global sum.
    """
    n = len(worker_data_list)
    chunk_size = len(worker_data_list[0]) // n
    # TODO: implement ring all-reduce
    pass


# ---- Test harness ----
if __name__ == "__main__":
    np.random.seed(0)
    N_WORKERS = 4
    VECTOR_SIZE = 8

    worker_data = [np.random.randint(1, 10, size=VECTOR_SIZE).astype(float)
                   for _ in range(N_WORKERS)]

    print("Worker gradients:")
    for i, d in enumerate(worker_data):
        print(f"  Worker {i}: {d}")

    expected_sum = sum(worker_data)
    print(f"\nExpected global sum: {expected_sum}")

    result_naive = simulate_allreduce_naive(worker_data)
    result_ring  = simulate_allreduce_ring(worker_data)

    print(f"\nNaive all-reduce Worker 0: {result_naive[0]}")
    print(f"Ring all-reduce Worker 0: {result_ring[0]}")

    assert np.allclose(result_naive[0], expected_sum), "Naive failed!"
    assert np.allclose(result_ring[0], expected_sum), "Ring failed!"
    print("\nAll assertions passed.")
```

Your Tasks

1. Complete all four functions and pass the test harness.
2. Count the number of **point-to-point messages** for naive all-reduce vs. ring all-reduce for n workers with vectors of size d. Express both as a formula in n and d.
3. For $n = 8$ workers and $d = 1{,}000{,}000$ floats (float32, 4 bytes each), how much total data (MB) is transmitted by each algorithm?
4. **Why is ring all-reduce considered *bandwidth-optimal*?** Write 2–3 sentences connecting your formula to this property.

1) Implementation result

All four functions were implemented successfully, and the test harness passed. Both naive all-reduce and ring all-reduce produced the correct global sum, followed by the message **"All assertions passed."**

2) Number of messages

For **naive all-reduce**:

- Messages = $2(n - 1)$
- Total data = $2(n - 1)d$

For **ring all-reduce**:

- Messages = $2n(n - 1)$
- Each message size = d/n

Total data:
$2n(n - 1) \times (d/n) = 2(n - 1)d$

3) Total data (n = 8, d = 1,000,000)

Each float32 = 4 bytes

Total data:
$2(8 - 1)(1{,}000{,}000) \times 4 = 56{,}000{,}000$ bytes

≈ **53.41 MB**

Both algorithms transmit approximately **53.41 MB**.

4) Why ring all-reduce is bandwidth-optimal

Ring all-reduce is bandwidth-optimal because it distributes communication evenly across all workers. Each worker sends only a fraction (d/n) of the total data in each step, avoiding a single bottleneck. This allows full utilization of network bandwidth and improves scalability compared to centralized approaches.

Task 3.2: Broadcast vs. Exchange Cost in Spark [6 pts]

Starter Code

```python
from pyspark.sql.functions import broadcast

# Small lookup table (fits in driver memory)
lookup = spark.createDataFrame(
    [(i, f"category_{i}") for i in range(100)],
    ["id", "category"])

# Large table
large = train_df.select(
    (F.col("age") % 100).alias("id"),
    F.col("label"))

# Without broadcast hint
t0 = time.time()
_ = large.join(lookup, on="id").count()
t_no_bc = time.time() - t0

# With broadcast hint
t0 = time.time()
_ = large.join(broadcast(lookup), on="id").count()
t_bc = time.time() - t0

print(f"Without broadcast: {t_no_bc:.3f}s")
print(f"With broadcast: {t_bc:.3f}s")
print(f"Speedup: {t_no_bc / t_bc:.2f}x")
```

Your Tasks

1. Run the code and record both times and the speedup.
2. Call .explain() on each joined DataFrame and identify the join strategy used in each case (BroadcastHashJoin vs. SortMergeJoin).
3. When should you **not** use a broadcast hint? Give two concrete conditions.

1) Execution time and speedup

- Without broadcast: **4.439 s**
- With broadcast: **1.852 s**
- Speedup: **2.40×**

2) Join strategy from explain()

From the execution plan:

- Without broadcast → **BroadcastHashJoin (BuildLeft)**
- With broadcast → **BroadcastHashJoin (BuildRight)**

This shows that Spark automatically selected a broadcast join in both cases because the lookup table is small, but the broadcast hint changed the build side.

3) When not to use broadcast

Broadcast should not be used in the following cases:

1. When the table being broadcast is too large to fit in memory, as this can cause memory pressure or crashes on the driver and executors.
2. When both tables are large, since broadcasting will not provide performance benefits and may increase overhead instead.

Grading Rubric

Task	Description	Max Points	Your Score
1.1	Memory profiling table and dtype analysis	10	
1.2	Amdahl's Law simulation and analysis	10	
2.1	Dataset load, label creation, class balance	8	
2.2	MLlib pipeline, evaluation, execution plan	14	
2.3	Partition scaling experiment and analysis	8	
3.1	Collective primitives implementation	12	
3.2	Broadcast vs. exchange in Spark	6	
Total		**68**	

Submit a **single ZIP file** named `Lab9A_YourID.zip` containing:

- `lab9a_answers.pdf` — your written answers and observations
- `memory_wall.py`, `amdahl.py`
- `spark_pipeline.py` — Tasks 2.1, 2.2, 2.3 in one script
- `collectives.py`

Deadline: One week from lab date. Late submissions lose 10 % per day.
Note: Lab 9B (Advanced Systems) builds directly on the Spark session and collectives from this lab.